

Transaction Management: Concurrency Control

How a DBMS runs hundreds of transactions at once while still guaranteeing the result each one would have gotten running alone.

Agenda

- 1 Why concurrent execution needs control: four anomalies
- 2 Serial vs. concurrent schedules, and a precedence-graph example
- 3 Lock-based concurrency control: shared & exclusive locks
- 4 Two-Phase Locking (2PL): growing & shrinking phases
- 5 Deadlocks: a concrete example, detection & prevention
- 6 The SQL standard's four isolation levels

The Need for Concurrency Control

Running transactions one at a time would trivially guarantee correctness — but every user would wait in a single queue. So a DBMS interleaves operations from multiple transactions for throughput. Left unchecked, this opens the door to four well-known anomalies.

● **Lost Update**

Two transactions overwrite each other's work.

● **Dirty Read**

A read of an uncommitted, later-rolled-back value.

● **Non-Repeatable Read**

Same row, two different values, one transaction.

● **Phantom Read**

Same query, a different set of rows.

Anomaly 1: Lost Update

Account A101, balance 1000. T1 deposits 100; T2 deposits 50. Run serially, the final balance must be 1150.

Time	T1	T2	balance
t1	READ balance → 1000		1000
t2		READ balance → 1000	1000
t3		WRITE balance = 1000+50 = 1050	1050
t4		COMMIT	1050
t5	WRITE balance = 1000+100 = 1100		1100
t6	COMMIT		1100

Result: 1100, not 1150. T1 read the original 1000 before T2's deposit happened; its write at t5 overwrites T2's committed +50 entirely.

Anomaly 2: Dirty Read

A transaction reads a value written by another transaction that has not yet committed — and that transaction later rolls back.

Time	T1	T2	balance
t1	WRITE balance = 1000+100 = 1100 (uncommitted)		1100
t2		READ balance → 1100	1100
t3		(uses 1100 in further calculation)	1100
t4	ROLLBACK (deposit reversed)		1000

Result: T2 acted on data that never existed. T2 read and acted on 1100 — a value that, once T1 rolled back, never became a real, committed fact about this account. Anything T2 computed from that read is based on data that formally never existed.

Anomaly 3: Non-Repeatable Read

A transaction reads the same row twice and gets two different values, because another transaction committed a change in between.

Time	T1	T2	balance
t1	READ balance → 1000		1000
t2		WRITE balance = 1000+100 = 1100	1100
t3		COMMIT	1100
t4	READ balance → 1100 (same txn, different value!)		1100

Result: T1's own view of the data changed mid-transaction. Unlike a dirty read, the value T2 wrote is legitimately committed — the problem is only that T1 read the same row twice and got two different, both-valid, answers.

Anomaly 4: Phantom Read

A transaction re-runs the same query and finds a different set of rows, because another transaction inserted or deleted matching rows in between.

Time	T1	T2
t1	SELECT * FROM Account WHERE balance > 5000 → 3 rows	
t2		INSERT INTO Account VALUES ('A999', 9000)
t3		COMMIT
t4	(same query re-run) → 4 rows	

Result: a new row appears inside T1's query range. No row T1 already saw was changed — the anomaly is an entirely new row appearing, which a simple row-level lock cannot prevent, because there was no existing row to lock at t1.

Schedules and Serializability

A schedule is the actual sequence in which operations of one or more transactions execute. A concurrent schedule is serializable if its effect is equivalent to some serial schedule of the same transactions — all four anomalies above are exactly the signature of a schedule that is NOT serializable.

● Serial Schedule

T1 fully completes, then T2 fully completes (or vice versa) — always correct, no overlap.

● Serializable Concurrent Schedule

Operations interleaved for speed, but the result matches some serial ordering — the goal.

Conflict serializability: two operations “conflict” if they access the same data item and at least one is a write; a schedule is conflict-serializable if its conflicting operations can be reordered, without changing their relative order, into a valid serial schedule.

WORKED EXAMPLE

Worked Example: A Concrete Schedule

T1 = Transfer A101 → A205 (read, write A101; then read, write A205). T2 = Transfer A205 → A101 (read, write A205; then read, write A101) — running concurrently, each touching the other transaction's account in reverse order.
Same T1/T2 cross-transfer as the deadlock example later in this lecture — here, with no locking at all.

Time	T1	T2
t1	R1(A101)	
t2	W1(A101)	
t3		R2(A205)
t4		W2(A205)
t5	R1(A205)	
t6	W1(A205)	
t7		R2(A101)
t8		W2(A101)

Remember: two operations conflict only if they touch the same data item and at least one is a write. Next: trace every conflict on A101, then on A205.

Deriving Conflict Edges — From A101

Four operations touch A101: t1 R1(A101), t2 W1(A101), t7 R2(A101), t8 W2(A101). Of the 6 possible pairings among them, the 2 same-transaction pairs (t1,t2 and t7,t8) can't create a precedence edge — only cross-transaction pairs can. That leaves 4 pairs to check:

Pair	Order on A101	Conflict?	Edge
(t1, t7)	R1(A101), R2(A101) — both reads	No conflict	—
(t1, t8)	R1(A101) before W2(A101)	Conflicts (read-write)	T1 → T2
(t2, t7)	W1(A101) before R2(A101)	Conflicts (write-read)	T1 → T2
(t2, t8)	W1(A101) before W2(A101)	Conflicts (write-write)	T1 → T2

Conclusion: Every conflicting pair on A101 agrees — edge T1 → T2.

Deriving Conflict Edges — From A205

Four operations touch A205: t3 R2(A205), t4 W2(A205), t5 R1(A205), t6 W1(A205). Again, only the 4 cross-transaction pairings can create an edge — the pairs (t3,t4) and (t5,t6) are each within a single transaction and don't count:

Pair	Order on A205	Conflict?	Edge
(t3, t5)	R2(A205), R1(A205) — both reads	No conflict	—
(t3, t6)	R2(A205) before W1(A205)	Conflicts (read-write)	T2 → T1
(t4, t5)	W2(A205) before R1(A205)	Conflicts (write-read)	T2 → T1
(t4, t6)	W2(A205) before W1(A205)	Conflicts (write-write)	T2 → T1

Conclusion: Every conflicting pair on A205 agrees — edge T2 → T1.

The Precedence Graph: A Cycle Means Not Serializable

Combine both items' edges into one precedence graph — a node per transaction, an edge per conflict:



Not conflict-serializable: Combining both items' edges gives $T1 \rightarrow T2$ (from A101) and $T2 \rightarrow T1$ (from A205) — a cycle. A conflict-serializable schedule's precedence graph must be acyclic; this one is not. No serial ordering of T1 and T2 could have produced this result — exactly what concurrency control exists to rule out.

Next: the Lock-Based Concurrency Control section shows exactly how locking would have stopped this interleaving from happening in the first place.

Lock-Based Concurrency Control

●

Shared Lock (S-lock)

Acquired to read a data item. Multiple transactions may hold a shared lock on the same item simultaneously — concurrent reads don't conflict.

●

Exclusive Lock (X-lock)

Acquired to write a data item. Only one transaction may hold it at a time, and no other transaction may hold any lock while it's held.

Lock compatibility matrix

Requested \ Held by another txn	Shared (S)	Exclusive (X)
Shared (S)	Compatible — both proceed	Must wait
Exclusive (X)	Must wait	Must wait

S

shared

X

exclusive

Two-Phase Locking (2PL)

Requiring locks alone isn't enough — a transaction that releases a lock early and re-acquires another later can still produce a non-serializable schedule. 2PL adds one rule: once a transaction starts releasing locks, it may never acquire another.

Growing Phase

May acquire new locks. May NOT release any lock yet.

Lock Point

Instant of maximum locks held — the boundary between the two phases.

Shrinking Phase

May release locks. May NOT acquire any new lock from here on.

2PL Preventing the Lost Update

Time	T1	T2	balance
t1	Acquire X-lock(balance)		—
t2	READ balance → 1000	BLOCKED — requests X-lock, must wait	1000
t3	WRITE balance = 1100	waiting	1100
t4	COMMIT, release X-lock	waiting	1100
t5		Acquire X-lock(balance)	—
t6		READ balance → 1100	1100
t7		WRITE balance = 1100+50 = 1150	1150
t8		COMMIT, release X-lock	1150

Result: 1150, matching a genuine serial execution — T2 waits from t2 to t5, the cost of a guaranteed-correct result.

Locks also stop the cross-transfer cycle from forming: see the worked precedence-graph example.

Deadlocks

A deadlock occurs when two (or more) transactions each hold a lock the other needs, and each is waiting for the other to release it — neither can ever proceed. 2PL guarantees serializability, not deadlock-freedom.

● T1

Holds X-lock(A101). Now requests X-lock(A205) — held by T2. T1 waits.

● T2

Holds X-lock(A205). Now requests X-lock(A101) — held by T1. T2 waits.

T1: X-lock(A101)

T2: X-lock(A205)

T1: ... requests X-lock(A205)

T2: ... requests X-lock(A101)

-> BLOCKED, waits on T2

-> BLOCKED, waits on T1

Worked example: this same T1/T2 cross-transfer, run with no locking, yields a non-serializable schedule — see the precedence-graph slides earlier in this lecture.

Resolving Deadlocks

● Detection

The DBMS periodically builds a wait-for graph (a node per transaction, an edge $T_i \rightarrow T_j$ when T_i waits on a lock T_j holds) and checks it for cycles. A cycle IS a deadlock; the DBMS aborts a victim transaction to break it.

● Prevention

Avoid the possibility entirely: order lock requests globally so a cycle can never form, or use timeouts — a transaction waiting too long is assumed deadlocked and aborted.

Detection always costs a rollback; prevention costs concurrency. Most production DBMSs use detection with a short timeout, since real deadlocks are rare relative to the overhead of strict prevention.

Isolation Levels

The SQL standard defines four isolation levels, each permitting more concurrency (and more anomalies) than the one below it — applications choose the weakest level still safe for their needs.

Read Uncommitted

Weakest — may read another transaction's uncommitted writes.

Read Committed

Only ever reads committed data — but a re-read can still change.

Repeatable Read

Re-reading the same row always returns the same value.

Serializable

Strongest — behaves exactly as if every transaction ran alone.

Isolation Levels vs. Anomalies Prevented

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	Possible	Possible	Possible
Read Committed	Prevented	Possible	Possible
Repeatable Read	Prevented	Prevented	Possible
Serializable	Prevented	Prevented	Prevented

Reading the table by column tells the real story: dirty reads are cheap to rule out and every level above the weakest does so; non-repeatable reads need locks held on rows already read for the whole transaction, which is why only Repeatable Read and above prevent them; phantom reads need locks on an entire range of potential rows, which only true Serializable isolation guarantees.

Lost update isn't in this table by design — it's prevented as a side effect of the locking each level uses; under 2PL, Repeatable Read and Serializable both hold write locks long enough to rule it out entirely.

Isolation and Consistency Are Different Questions

● Consistency (Lecture 29)

Asks whether a single transaction's final state obeys every integrity constraint.

● Isolation (this lecture)

Asks whether concurrently running transactions interfere with each other's intermediate, in-progress work.

A schema can be perfectly consistent and still suffer a lost update if isolation is too weak — the balance in the lost-update example never violated any CHECK constraint at any point; the row was simply wrong because two transactions stepped on each other. Choosing an isolation level is an explicit trade-off between correctness guarantees and concurrent throughput.

Key Takeaways

- Uncontrolled interleaving produces four anomalies: lost update, dirty read, non-repeatable read, and phantom read.
- A schedule is serializable if its result matches some serial execution of the same transactions.
- Build a precedence graph from a schedule: one edge $T_i \rightarrow T_j$ per conflict. A cycle means the schedule is NOT conflict-serializable.
- Two-Phase Locking (2PL) — a growing phase, then a shrinking phase — guarantees serializability; it fixed the lost-update schedule directly.
- Deadlock occurs when transactions wait on each other in a cycle; resolved by detection (wait-for graph) or prevention (ordering, timeouts).
- The SQL standard's four isolation levels trade off concurrency against which anomalies they prevent.